Київський національний університет
Імені Тараса Шевченка.
Кафедра: Мережевих та інтернет-технологій.

Лабораторна робота №6

з дисципліни: Системне Програмування
На тему: “Файлове Введення–Виведення”

Студента 4 курсу:
Групи МІТ-41
Мулико Володимира

Київ - 2025р.

Завдання 1. Візуалізація ієрархії директорій

Розробити консольну програму, яка приймає від користувача шлях до директорії та рекурсивно виводить на екран повну ієрархічну структуру всіх вкладених файлів та папок.

Код:

```
using System;

using System.IO;



class Program

{

   static void Main(string[] args)

   {

       Console.Write("Введіть шлях до директорії: ");

       string path = Console.ReadLine() ?? "";



       if (Directory.Exists(path))

           PrintDirectory(path, 0);

       else

           Console.WriteLine("Директорія не знайдена.");

   }



   static void PrintDirectory(string path, int indent)

   {

       var dirInfo = new DirectoryInfo(path);

          Console.WriteLine(new string(' ', indent * 2) + "\U0001F4C1
Folder" + dirInfo.Name);
```

```
        try
        {
            foreach (var file in dirInfo.GetFiles())
                Console.WriteLine(new string(' ', (indent + 1) * 2) +
"\U0001F4C4 File" + file.Name);


            foreach (var dir in dirInfo.GetDirectories())
                PrintDirectory(dir.FullName, indent + 1);
        }
        catch (UnauthorizedAccessException)
        {
            Console.WriteLine(new string(' ', (indent + 1) * 2) + "Немає
доступу");
        }
        catch (Exception ex)
        {
                Console.WriteLine(new string(' ', (indent + 1) * 2) +
"Помилка: " + ex.Message);
        }
    }
}
```

Вивід:

```
[volodymyr@archlinux TASK1]$ dotnet run
Введіть шлях до директорії: /home/volodymyr/diploma_projct
```

```
📁 Folder Document_Folder
  📄 FileDiagramIO_WorkFlow_Mulyko.png
  📄 FileMulyko_Diploma.odt
```

Синопсис:

Програма призначена для сканування вказаної користувачем директорії та виводу її вмісту у вигляді деревоподібної структури. Основна складність задачі полягає в тому, що глибина вкладеності папок заздалегідь невідома, тому для розв’язання використано рекурсивний підхід.

Метод Directory.Exists(path): Використовується на самому початку для валідації введених даних. Оскільки в Linux (Arch Linux) шлях чутливий до регістру та символів, ця перевірка гарантує, що програма не почне роботу з неіснуючим об'єктом.

Рекурсивний метод PrintDirectory(path, indent): Це ядро програми. Він викликає сам себе для кожної знайденої піддиректорії.

```
foreach (var dir in dirInfo.GetDirectories())

                PrintDirectory(dir.FullName, indent + 1);
```

Параметр indent (відступ) збільшується при кожному зануренні глибше в ієрархію, що дозволяє візуально відобразити вкладеність.

Завдяки використанню простору імен System.IO, програма ефективно взаємодіє з ядром Linux для отримання метаданих файлів. Використання рекурсії забезпечує мінімальну кількість коду при

максимальній функціональності (обхід будь-якої глибини дерева каталогів).

Завдання 2. Рекурсивний пошук та аналіз метаданих файлів

Розробити консольну програму для середовища, яка виконує пошук файлу за заданим ім'ям (або його фрагментом) у вказаній директорії та всіх її піддиректоріях.

Код:

```
using System;

using System.IO;



class FileSearcher

{

   static void Main()

   {

       Console.Write("Введіть шлях для пошуку: ");

       string rootPath = Console.ReadLine() ?? "/home/volodymyr";

       Console.Write("Введіть ім'я файлу (повне або часткове): ");

       string fileName = Console.ReadLine() ?? "";



       SearchFiles(rootPath, fileName);

   }



   static void SearchFiles(string path, string target)

   {

       try
```

```
        {
                        foreach  (string  f  in  Directory.GetFiles(path,
$"*{target}*"))
            {
                FileInfo info = new FileInfo(f);
                Console.WriteLine($"\nЗнайдено: {info.FullName}");
                   Console.WriteLine($"Розмір: {info.Length / 1024.0:F2}
KB");
                Console.WriteLine($"Створено: {info.CreationTime}");
            }

            foreach (string d in Directory.GetDirectories(path))
                SearchFiles(d, target);
        }
        catch (UnauthorizedAccessException)
        {
            Console.WriteLine($"Немає доступу: {path}");
        }
        catch (DirectoryNotFoundException)
        {
            Console.WriteLine($"Директорію не знайдено: {path}");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Помилка в {path}: {ex.Message}");
        }
```

```
    }

}
```

Синопсис:

Програма побудована на базі класу FileSearcher і використовує методи простору імен System.IO для взаємодії з дисковою підсистемою Linux. Основний акцент зроблено на поєднанні рекурсивного обходу дерева каталогів із застосуванням фільтрів пошуку.

Метод Directory.GetFiles(path, $"*{target}*"):

Цей рядок виконує роль пошукового рушія. Використання символів підстановки (*) дозволяє знаходити файли, навіть якщо користувач ввів лише частину назви (наприклад, "entropy" замість "experiment1_entropy.py").

Робота з класом FileInfo:

Замість простого виводу назви, програма створює об'єкт FileInfo. Це дозволяє отримати доступ до "паспорта" файлу:

```
FileInfo info = new FileInfo(f);

               Console.WriteLine($"\nЗнайдено: {info.FullName}");

                  Console.WriteLine($"Розмір: {info.Length / 1024.0:F2}
KB");

               Console.WriteLine($"Створено: {info.CreationTime}");
```

Рекурсивне занурення через Directory.GetDirectories:

Програма не обмежується поточною папкою. Вона збирає список усіх підпапок і для кожної з них повторно викликає метод пошуку, що дозволяє знайти файл, навіть якщо він захований глибоко в структурі проєкту.

Вивід:

```
[volodymyr@archlinux TASK2]$ dotnet run
Введіть шлях для пошуку: /home/volodymyr/diploma_project
Введіть ім'я файлу (повне або часткове): entropy
```

```
Знайдено: /home/volodymyr/diploma_project/Stage4/.venv/lib64/python3.9/site-packages/scipy/stats/tests/__pyc
ache__/test_entropy.cpython-39.pyc
Розмір: 10.09 KB
Створено: 2/11/2026 9:23:17 PM

Знайдено: /home/volodymyr/diploma_project/Stage4/.venv/lib64/python3.9/site-packages/dns/entropy.py
Розмір: 4.66 KB
Створено: 2/11/2026 9:16:02 PM

Знайдено: /home/volodymyr/diploma_project/Stage4/.venv/lib64/python3.9/site-packages/dns/__pycache__/entropy
.cpython-39.pyc
Розмір: 3.33 KB
Створено: 2/11/2026 9:16:02 PM

Знайдено: /home/volodymyr/diploma_project/__pycache__/experiment1_entropy.cpython-39.pyc
Розмір: 3.67 KB
Створено: 2/11/2026 9:22:28 PM

Знайдено: /home/volodymyr/diploma_project/__pycache__/experiment1_entropy.cpython-310.pyc
Розмір: 3.69 KB
Створено: 2/11/2026 5:35:26 PM
[volodymyr@archlinux TASK2]$
```

Завдання 3. Графічний файловий менеджер з функцією перегляду вмісту

Розробити повноцінний графічний застосунок для ОС Arch Linux, що реалізує функціонал провідника.

Код файлу MainWindows.axaml

```
<Window xmlns="https://github.com/avaloniaui"

       xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

       x:Class="FileManagerApp.MainWindow"

       Title="Arch File Explorer" Width="900" Height="600">

   <DockPanel Margin="10">

             <StackPanel  DockPanel.Dock="Top"  Orientation="Horizontal"
Spacing="5" Margin="0,0,0,10">

           <Button Content="⬅" Click="GoUp_Click"/>
```

```
                                <TextBox   Name="PathBox"   Width="700"
Text="/home/volodymyr/diploma_project"/>

            <Button Content="Оновити" Click="Refresh_Click"/>

        </StackPanel>



        <Grid ColumnDefinitions="300, *">

                            <ListBox  Name="FileListView"  Grid.Column="0"
DoubleTapped="FileListView_DoubleTapped"/>



                        <ScrollViewer  Grid.Column="1"  Margin="10,0,0,0"
Background="#f0f0f0">

                    <TextBlock Name="FileContentView" TextWrapping="Wrap"
FontFamily="Monospace" Padding="10"/>

            </ScrollViewer>

        </Grid>

    </DockPanel>

</Window>
```

Код файлу [MainWindow.axaml.cs]

```
using Avalonia.Controls;

using Avalonia.Interactivity;

using Avalonia.Input;

using System;

using System.IO;

using System.Collections.Generic;

using System.Linq;
```

```
namespace FileManagerApp;


public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        string defaultPath = "/home/volodymyr/diploma_project";
            LoadDirectory(Directory.Exists(defaultPath) ? defaultPath :
Environment.GetFolderPath(Environment.SpecialFolder.UserProfile));
    }


    private void LoadDirectory(string? path)
    {
            if (string.IsNullOrEmpty(path) || !Directory.Exists(path))
return;


        PathBox.Text = path;
        var items = new List<string>();
        try
        {
            foreach (var dir in Directory.GetDirectories(path))
                items.Add("📁 " + Path.GetFileName(dir));
            foreach (var file in Directory.GetFiles(path))
                items.Add("📄 " + Path.GetFileName(file));
```

```
            FileListView.ItemsSource = items;

        }

           catch (Exception ex) { FileContentView.Text = "Помилка: " +
ex.Message; }

    }



            private void FileListView_DoubleTapped(object sender,
TappedEventArgs e)

    {

                if (FileListView.SelectedItem is string selected &&
!string.IsNullOrEmpty(PathBox.Text))

        {

            string name = selected.Substring(3);

            string newPath = Path.Combine(PathBox.Text, name);



            if (Directory.Exists(newPath))

                LoadDirectory(newPath);

            else if (File.Exists(newPath))

            {

                try {

                    FileContentView.Text = File.ReadAllText(newPath);

                } catch {

                    FileContentView.Text = "Неможливо прочитати файл.";

                }

            }
```

```
        }

    }



    private void GoUp_Click(object sender, RoutedEventArgs e)

    {

        if (!string.IsNullOrEmpty(PathBox.Text))

        {

            var parent = Directory.GetParent(PathBox.Text);

            if (parent != null) LoadDirectory(parent.FullName);

        }

    }



     private void Refresh_Click(object sender, RoutedEventArgs e) =>
LoadDirectory(PathBox.Text);

}
```

Синопсис

Програма побудована за принципом розділення декларативного опису інтерфейсу (XAML) та імперативної логіки обробки подій (C# Code-behind). Основна ідея полягає у створенні дворівневого вікна (Split-view), де ліва панель відповідає за навігацію (Master), а права — за відображення вмісту обраного об'єкта (Detail).

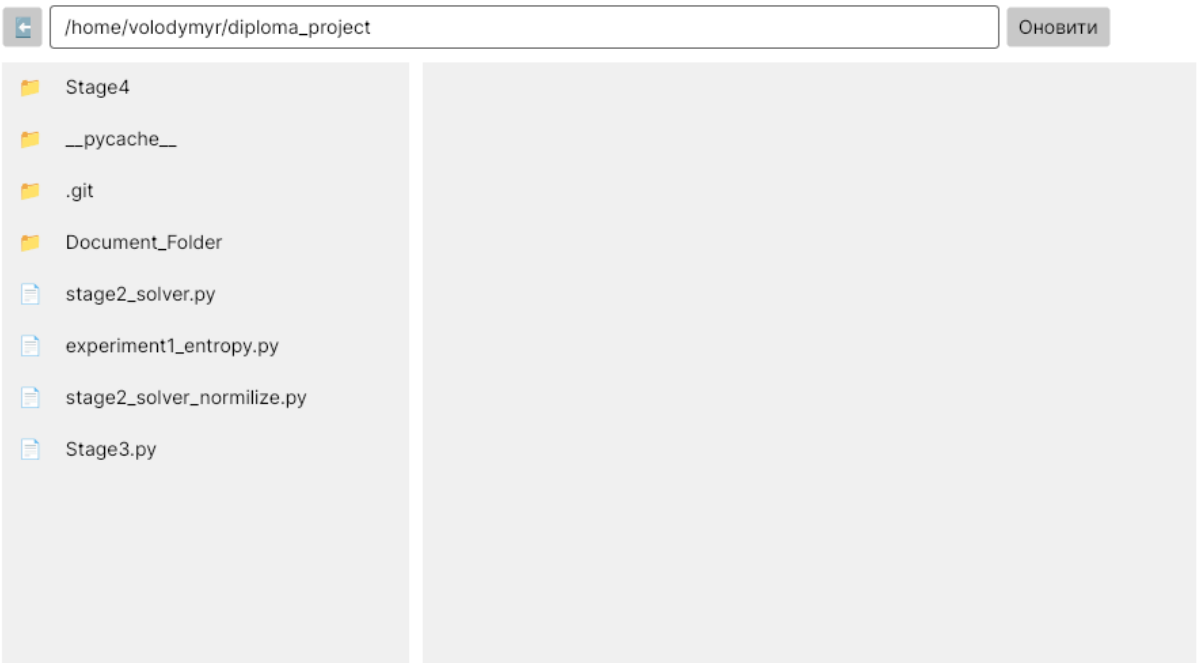
Динамічне оновлення інтерфейсу (LoadDirectory): Метод виконує роль центрального вузла оновлення даних. Він очищує попередній список і за допомогою методів Directory.GetDirectories та Directory.GetFiles формує нову колекцію рядків. Використання префіксів (📁 та 📄) дозволяє візуально розділити типи вузлів у списку ListBox.

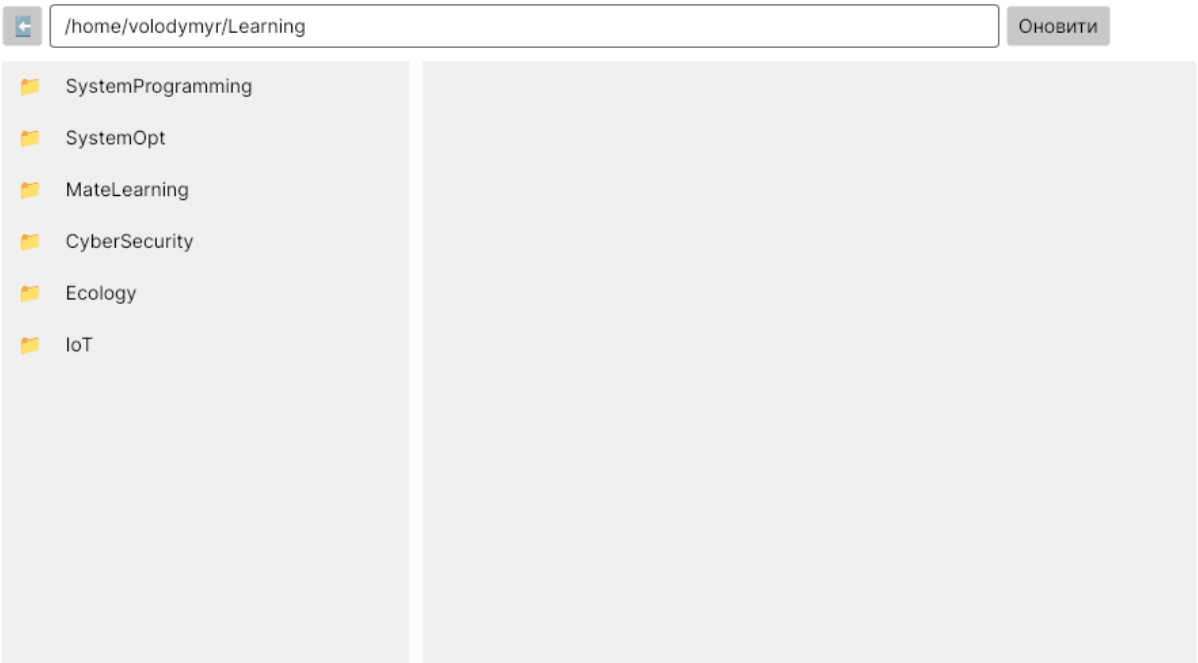
Інтерактивна навігація (FileListView_DoubleTapped): Це основний обробник подій (Event Handler). Код використовує механізм String Parsing (selected.Substring(3)), щоб відокремити іконку від назви об'єкта. Далі за допомогою Path.Combine програма безпечно будує новий шлях до файлу або папки, що дозволяє реалізувати логіку "занурення" в ієрархію директорій.

Робота з текстовим контентом (File.ReadAllText): Програма реалізує функцію швидкого перегляду (Quick Look). Якщо обраний елемент є файлом, він зчитується безпосередньо у текстове поле FileContentView. Використання властивостей TextWrapping="Wrap" та FontFamily="Monospace" у XAML забезпечує зручне читання вихідного коду (наприклад, твоїх Python-скриптів для диплома).

Логіка навігації вгору (GoUp_Click): Реалізована через звернення до статичного методу Directory.GetParent. Це дозволяє програмі коректно обробляти кореневі папки та забезпечує логічне переміщення назад по ієрархії каталогів Linux.

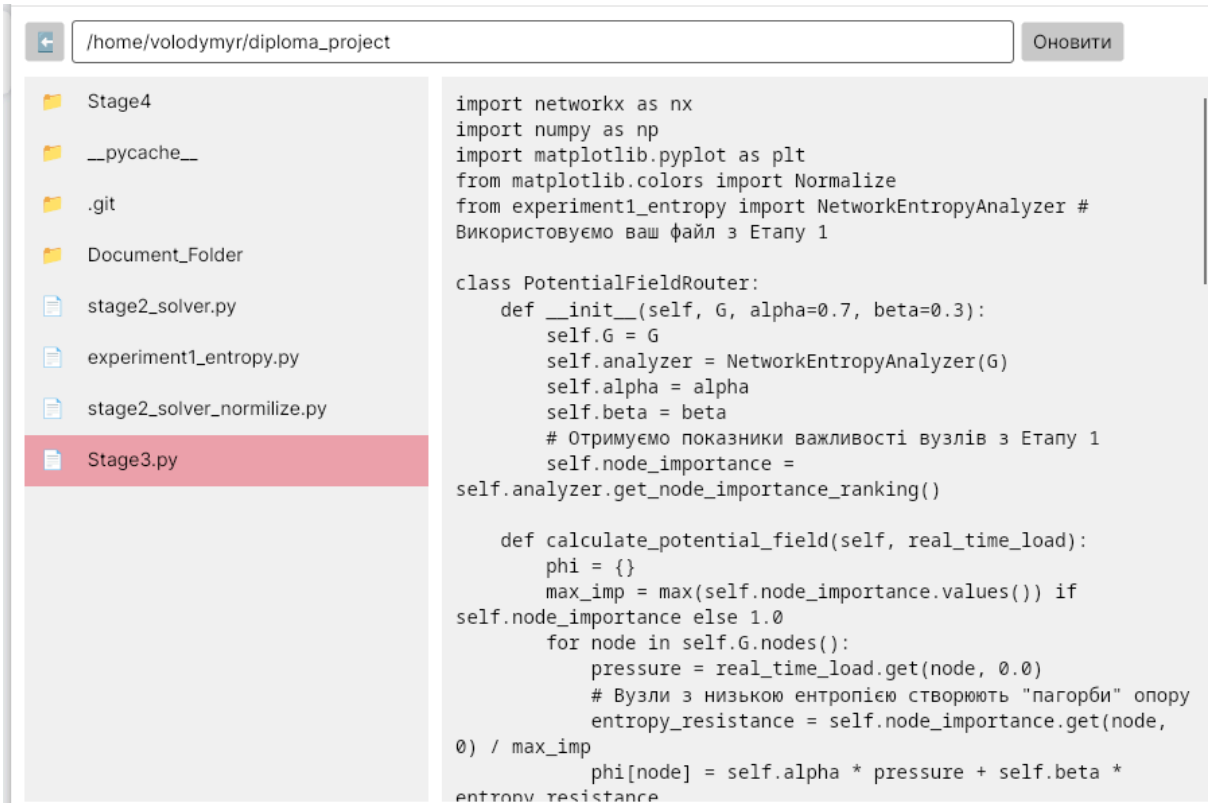
Вивід:





Просте читання директорій.

Функціонал кнопки “ ← ”



Фцункціонал читання файлу

Висновок:

Під час виконання лабораторної роботи було проведено комплексне дослідження методів взаємодії з файловою системою в середовищі .NET на базі операційної системи Arch Linux. Основний акцент було зроблено на застосуванні рекурсивних алгоритмів для обходу ієрархічних структур папок, реалізації гнучких механізмів пошуку файлів за шаблонами та вилученні метаданих об'єктів. Використання системних просторів імен, зокрема System.IO, дозволило ефективно обробляти шляхи до файлів, враховуючи специфіку прав доступу в Linux та забезпечуючи стабільну роботу програм навіть при виникненні виключних ситуацій, таких як відсутність доступу до захищених директорій.

Особлива увага була приділена розробці графічного інтерфейсу на базі фреймворку Avalonia UI, що дозволило трансформувати лінійну консольну логіку у повноцінний подієво-орієнтований застосунок. Реалізація функціоналу провідника з можливістю динамічної навігації через подвійний клік та інтегрованою областю попереднього перегляду

тексту продемонструвала переваги поєднання декларативної розмітки XAML із потужною логікою обробки подій на C#. Це дозволило створити інструмент, який є не лише навчальним прикладом, а й практичним помічником для візуального аналізу вихідного коду та структури дипломного проєкту.

У результаті роботи було закріплено навички побудови кросплатформних інтерфейсів та роботи з потоками даних. Розроблені програми підтвердили ефективність використання спецсимволів Unicode для візуалізації ієрархій та продемонстрували високий рівень відмовостійкості системи. Отриманий досвід є критично важливим для подальшої розробки складного програмного забезпечення, де вимагається глибока інтеграція з операційною системою та створення інтуїтивно зрозумілих засобів взаємодії з користувачем.